Fakultät für Informatik und Mathematik
Dr. Armin Größlinger
WS 2026/2027

Exercises for Lecture: Virtual Machines and Runtime Systems

Exercise Sheet 3 (Exceptions, Memory Management)

Problem 1 (Exceptions)

(a) With a dynamic management for exception handling, entering a `try` block can be accomplish in constant time (instead of time linear in the number of `catch` clauses). How can this be achieved?

(b) Finding the handler when an exception occurs can be implemented naïvely with time complexity $O(d\,n)$ for the static method, where d is the depth of the call stack and n the total number of `catch` clauses in the program. Devise a procedure to achieve a better time complexity (in common programs). What is the theoretically best time complexity possible?

(c) The lecture only discussed `catch` clauses. How can `finally` clauses be emulated (by rewriting a program containing `finally` clauses to an equivalent one not containing `finally` clauses)? Sketch your approach using pseudo code (e.g., with Java-like syntax and `goto` statements).

Problem 2 (Memory Management)

In Stud.IP, you can find the skeleton `mymalloc.c` for a memory management library. Using the `LD_PRELOAD` mechanism, one can load a dynamic library at the start of a program and replace symbols that come, for example, from the standard C library. See `mymalloc.c` for details how to use this mechanism. Implement in `mymalloc.c` the functions `malloc`, `calloc`, `realloc` and `free` using `sbrk`. (Further allocation functions such as `memalign` are implemented in `mymalloc.c` through `malloc` already, i.e., these functions need not be implemented.)

(a) Disregard freeing of allocated memory, i.e., `free` does nothing and in `malloc` simply more memory is requested using `sbrk`.

(b) (More complex, optional:) Implement a management of the allocated memory in, e.g., a linked list such that memory blocks deallocated by calling `free` are marked as available and subsequent calls to `malloc` search the list for a free block (of sufficient size); allocation of new memory using `sbrk` is only performed when no suitable block can be found.

Pay attention to align memory addresses returned by `malloc`, etc. at a 16 byte boundary to satisfy the common alignment requirements. You can use the functions `memset` and `memcpy` from `stdlib.h` if you need then (see the man pages on how to use them).

You can hand-in your programs by uploading them in the folder "Submissions" in Stud.IP; upload source files only. Hand-in is optional.

Due date: Thursday, November 05 at 18:00, discussion on November 06, 2026